

LogMyLife V1

Notification System — Corrected v1.1

FINAL ANTIGRAVITY IMPLEMENTATION PATCH

WHAT CHANGED

The original notification specification was structurally sound. This v1.1 closes four implementation gaps: 1) closed-app Android action execution, 2) Late Start vs Schedule Shift precedence, 3) exact domain-command payload contracts, and 4) deterministic Undo Shift using RescheduleEvent history.

SOURCE-OF-TRUTH RULE

This is an amendment to the previous LogMyLife V1 Notification System specification. Where this document conflicts with an earlier notification rule, v1.1 wins. The existing Planner/Scheduling specification remains authoritative for ActivityOccurrence and RescheduleEvent semantics.

PROBLEM CLOSED

A notification action cannot depend on the React Native UI/JS process being alive. The action must execute through an Android background-safe path, persist state transactionally, and then reconcile the planner.

Required execution path

```
Android Notification Action
|
▼
BroadcastReceiver / notification action handler
|
├─ validate action token + current state
▼
Native command dispatcher
|
▼
Local database transaction
(ActivityOccurrence / TimerSession / RescheduleEvent)
|
▼
Scheduling + notification reconciliation
|
├─ update/cancel stale notification
├─ post new notification if required
|
▼
Return success
├─ launch UI only when visual interaction is required
```

RECOMMENDED V1 IMPLEMENTATION

For local-state actions such as `START_TIMER`, `STOP_TIMER`, `RESUME_TIMER`, `FINISH_ACTIVITY`, `SKIP_ACTIVITY`, execute the command from a native/background-safe Android handler against the local DB. Do not require the React Native UI to boot first. Headless JS may be used for logic that genuinely belongs in JS, but it must not be the only guarantee for critical state mutation.

ANDROID 12+ RULE

Do not launch an Activity merely to process a notification action. Background activity-launch restrictions apply. Use a `BroadcastReceiver`/service-safe path for command execution; launch UI only for actions such as View Planner or other interactions that genuinely need a screen.

START_TIMER closed-app contract

```
Input:
command = START_TIMER
occurrenceId = A0-2026-08-17-DSA-001
actionId = notification-action-UUID
issuedAt = 2026-08-17T14:05:12+05:30
```

Handler:

1. Load ActivityOccurrence by occurrenceId.
2. Verify it is still startable.
3. If active TimerSession exists -> idempotent success.
4. Otherwise create TimerSession with actualStart = now.
5. Create RescheduleEvent if cascade is required.
6. Commit transaction.
7. Reconcile notifications.
8. Return success.

CRASH SAFETY

The command must be atomic. If the process dies after the DB transaction commits but before the notification update, the next reconciliation pass must repair the notification. Never leave the database in a half-created timer state.

02

Late Start vs Schedule Shift — Precedence & Deduplication

SINGLE NOTIFICATION RULE

A single late start must not produce two notifications containing the same schedule-shift information. The notification engine chooses one presentation based on the scope of the resulting change.

Precedence matrix

Only current activity timing changes	YES	NO	Late Start
Current + exactly 1 downstream movable activity shifts	YES	NO	Late Start + concise next-activity detail
Current + 2+ downstream activities shift	NO	YES	Schedule Shift summary
Late start causes fixed-event conflict	NO	YES	Conflict notification
No schedule change because task runs in place	YES	NO	Late Start

DEDUP KEY

Generate a deterministic notification event key from the domain event, e.g.

late_start:{occurrenceId}:{rescheduleEventId} or schedule_shift:{rescheduleEventId}. Persist delivered event keys so retries cannot produce duplicates.

Example:

DSA 2:00–3:00 → starts 2:20

Project 3:00–4:00 → 3:20–4:20

Workout 4:00–5:00 → 4:20–5:20

2 downstream activities shifted:

→ ONE Schedule Shift notification

→ NO separate Late Start notification

Only Project shifts:

→ ONE Late Start notification

→ include “Project moved to 3:20” inside it.

03

Domain Command Payload Contracts

CONTRACT RULE

Every notification action carries a stable occurrenceId or other domain identifier. Never identify an activity by title, list index, UI position or notification text.

START_TIMER	occurrenceId, actionId, issuedAt	Creates/returns active TimerSession
STOP_TIMER	occurrenceId, timerSessionId, actionId, issuedAt	Stops active TimerSession
RESUME_TIMER	occurrenceId, timerSessionId, actionId, issuedAt	Resumes paused TimerSession
PAUSE_TIMER	occurrenceId, timerSessionId, actionId, issuedAt	Pauses TimerSession
FINISH_ACTIVITY	occurrenceId, timerSessionId?, actionId, issuedAt	Marks occurrence completed by policy
SKIP_ACTIVITY	occurrenceId, actionId, issuedAt, reason?	Marks occurrence skipped
RESCHEDULE_ACTIVITY	occurrenceId, requestedStart, requestedEnd?, actionId, issuedAt, reason	Creates RescheduleEvent + cascade
UNDO_RESCHEDULE	rescheduleEventId, actionId, issuedAt	Reverts safe effects
ADD_REMAINING_TIME	occurrenceId, remainingMinutes, requestedStart?, actionId, issuedAt	Creates continuation
RESOLVE_CONFLICT	conflictId, resolutionType, targetOccurrenceId?, requestedStart?, actionId, issuedAt	Applies explicit resolution
VIEW_ACTIVITY	occurrenceId	Opens detail
VIEW_PLANNER	date?	Opens planner
VIEW_TIMELINE	date?, from?, to?	Opens timeline
ENABLE_TRACKING	source='notification'	Opens Usage Access settings
ENABLE_NOTIFICATIONS	source='notification'	Opens notification settings

Core model mapping

```
ActivityOccurrence
id
plannedStart
plannedEnd
duration
status
timerRequired
fixed
parentActivityId

TimerSession
id
occurrenceId
actualStart
actualEnd
accumulatedActiveDuration
status

RescheduleEvent
id
causeOccurrenceId
createdAt
reason
oldStart
oldEnd
newStart
newEnd
affectedOccurrenceIds[]
inverseAppliedAt?
```

IDEMPOTENCY

actionId must be unique per notification action attempt. Persist command execution/result metadata or otherwise guarantee that retrying the same action cannot create a second TimerSession, second RescheduleEvent or duplicate completion transition.

04 Undo Shift — Deterministic RescheduleEvent Algorithm

SOURCE OF TRUTH

Undo is based on the persisted RescheduleEvent, not on current visible planner positions. The event stores old/new intervals for every affected occurrence so the cascade can be reversed safely.

RescheduleEvent shape

```

RescheduleEvent {
  id
  causeOccurrenceId
  reason
  createdAt

  changes: [
    {
      occurrenceId,
      oldStart,
      oldEnd,
      newStart,
      newEnd
    }
  ]

  status: ACTIVE | PARTIALLY_UNDONE | UNDONE
  inverseAppliedAt?
}

```

Undo algorithm

```

UNDO_RESCHEDULE(rescheduleEventId):

1. Load RescheduleEvent R.
2. If R.status == UNDONE:
  return idempotent success.

3. Load every affected ActivityOccurrence.

4. For each change C:
  current = current interval of C.occurrenceId

5. SAFETY CHECK:
  Restore C.oldStart/C.oldEnd only if current still
  represents the schedule produced by R
  (current == C.newStart/C.newEnd, apart from metadata).

6. If user manually edited the occurrence after R:
  DO NOT overwrite it.
  Mark it protected/conflicted.

7. If occurrence already STARTED or COMPLETED after R:
  DO NOT rewrite actual history.
  Restore only safe future planning.

8. Apply safe inverse changes atomically.

9. Create an inverse/audit event referencing R.

10. Set R.status = UNDONE or PARTIALLY_UNDONE.

11. Re-run conflict detection + planner reconciliation.

12. Rebuild/cancel notifications affected by the undo.

13. Return:
  restored[]
  protected[]
  conflicts[]

```

EXAMPLE

Project 3:00–4:00 → shifted by R to 3:20–4:20. If still 3:20–4:20 and not started → restore safely. If user changed it to 3:45–4:45 → do not overwrite; mark protected. If it already started → never rewrite actual history; only safe future planning may be adjusted.

```

DOMAIN EVENT
|
▼
Notification Policy Resolver
|
|— current state valid? —NO—> suppress stale
|— duplicate event key? —YES—> suppress
|— precedence? —————> resolve Late Start/Shift
▼
Notification Record
|
|— type / priority / eventKey
|— occurrenceId / rescheduleEventId
|— actions[]
|
▼
Android Notification
|
|— action tap
|
▼
Domain Command
|
▼
Native/background-safe path
|
▼
Transactional DB update
|
▼
Scheduling/Reconciliation
|
▼
Notification reconciliation

```

FINAL ENGINEERING RULE

The notification system must never become a second scheduling engine. It only requests domain commands and renders domain state. Planner, Timer, Reconciliation and RescheduleEvent remain authoritative.

- ☐ 1. Kill the React Native UI process. Tap START_TIMER from notification. Verify TimerSession is created exactly once.
- ☐ 2. Tap START_TIMER twice rapidly. Verify exactly one active TimerSession.
- ☐ 3. Receive Upcoming DSA, start early, then verify Starting Now is suppressed.
- ☐ 4. Start DSA 20 minutes late with one downstream activity shifted. Verify only one Late Start notification.
- ☐ 5. Start DSA 20 minutes late with two downstream activities shifted. Verify only one Schedule Shift notification.
- ☐ 6. Cause a fixed-event conflict. Verify no fixed event is silently moved.
- ☐ 7. Execute RESCHEDULE_ACTIVITY and verify RescheduleEvent contains old/new intervals for every affected occurrence.
- ☐ 8. Undo a fresh RescheduleEvent. Verify safe future occurrences return to oldStart/oldEnd.
- ☐ 9. Manually edit one affected occurrence after rescheduling, then Undo. Verify it is protected and not overwritten.
- ☐ 10. Start an affected occurrence, then Undo. Verify actual history is untouched.
- ☐ 11. Crash after DB commit but before notification refresh. Verify reconciliation repairs notifications.
- ☐ 12. Replay the same actionId. Verify idempotent result and no duplicate state.
- ☐ 13. Reboot and verify active timer/planner state is reconstructed from persistence.
- ☐ 14. Verify data-only notification actions do not require launching an Activity.
- ☐ 15. Verify all 30 notification types obey precedence, cooldown, deduplication and state rules.

ANTIGRAVITY HANDOFF INSTRUCTION

Implement this v1.1 correction before treating the notification specification as final.

First inspect the existing codebase and previous Planner/Scheduling models. Reuse ActivityOccurrence and RescheduleEvent semantics. Implement the four corrected contracts above. Do not invent a second schedule model or a UI-only notification action path. Run the additional tests and report platform limitations explicitly.

Implementation references

Android notifications: <https://developer.android.com/develop/ui/views/notifications>

Background activity launch behavior: <https://developer.android.com/guide/components/activities/background-starts>

UsageStatsManager: <https://developer.android.com/reference/android/app/usage/UsageStatsManager>